Frontend Design

Programação Web 2

Objetivos de Aprendizagem

- Conhecer estratégias de desenvolvimento *Frontend*

Agenda

- *Design Patterns*
- Abordagens de desenvolvimento

Design Patterns

> Design **patterns** provide proven solutions to common problems in software development.

Design Patterns

Propriedades

- Estruturas escaláveis e reutilizáveis
- Melhoria da mantenabilidade do código
- Robustez, eficiência, etc

Design Patterns

Exemplos

- MVC (*Model-View-Controller*)
- Flux Architecture
- Dependency Injection
- Component-Based Architecture

MVC

> O padrão MVC separa uma aplicação em três componentes interconectados: Modelo, Visão e Controlador. O Modelo representa os dados e a lógica de negócio, a Visão lida com a apresentação ao usuário e o Controlador atua como intermediário entre Modelo e Visão

Flux Architecture

> Flux é um padrão de arquitetura de aplicações popularizada pelo Facebook (Meta). Enfatiza o fluxo de dados unidirecional força a separação estrita entre os componentes: ***Dispatchers*** (recebem ações que são disparadas aos *listeners*), ***Stores*** (armazenam dados que são modificados por *dispatchers*) e ***Views*** (Componentes React que escutam mudanças em *Stores* e chamam ações dos *dispatchers*).

Flux architecture cheatsheet

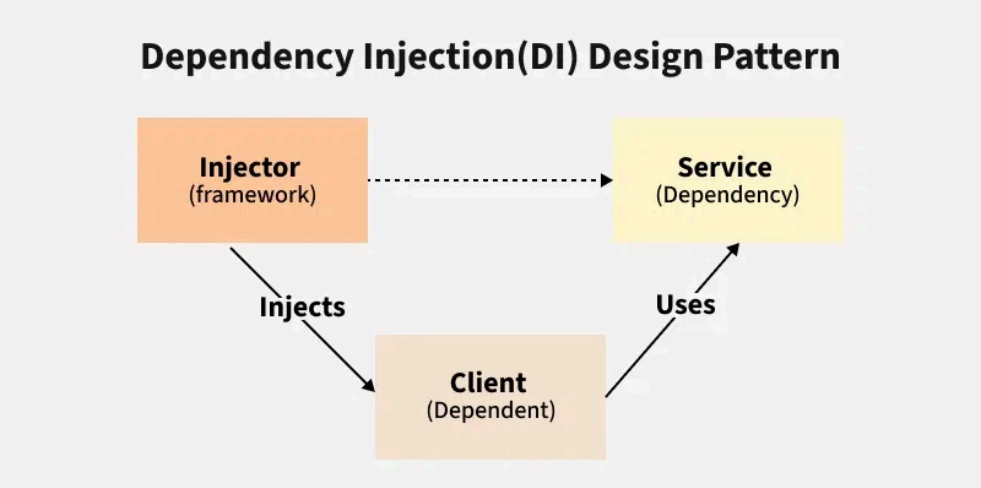
Dependency Injection

> Dependency Injection é um padrão de projeto que busca gerenciar dependências de componentes injetando-os a partir de fontes externas ao invés de criá-las dentro do próprio componente.

Dependency Injection

Objetivos

- Aumentar a flexibilidade
- Reduzir o acomplamento entre as classes
- Aumentar a reutilização de código
- Melhorar a manutenção e escalabilidade



Component Based

> Arquitetura baseada em componentes enfatiza a construção de aplicações pela composição de componentes de interface do usuário reutilizáveis. Estimula o encapsulamento, modularidade e a reutilização

Component Based

Vantagens

- **Reutilização** Os componentes podem ser facilmente reutilizados em diferentes partes da aplicação.
- **Encapsulamento** Cada componente é responsável pelo seu próprio estado e comportamento, aumentando a modularidade e a facilidade de manutenção do código.
- **Eficiência de desenvolvimento** As equipes podem trabalhar em componentes individuais simultaneamente, promovendo o desenvolvimento paralelo.

Component Based

Desvantagens

- **Complexidade** em projetos grandes. À medida que o número de componentes aumenta, o gerenciamento da comunicação e das dependências pode ser desafiador.
- Problemas de **desempenho**. O design inadequado de componentes ou a renderização excessiva podem afetar o desempenho.

Component-Based Architecture

> As bibliotecas modernas de *frontend* constroem apps utilizando a arquitetura baseada em componentes. React, Angular, Vue, Next e Svelte são exemplos de *frameworks* consagrados que utilizam dessa arquitetura.

Arquiteturas de desenvolvimento

Arquiteturas de desenvolvimento

- Static Site Generator (SSG)
- Single Page Application (SPA)
- Server-Side Rendering (SSR)

Single Page Application (SPA)

- O conteúdo é carregado por meio de arquivos Javascript para todo o aplicativo e hospedado em uma única página HTML
- Os arquivos Javascript abrigam todos os dados relativos à lógica da aplicação, UI e comunicação com o servidor
- À medida que o usuário realiza requisições a página vai sendo modificada sem necessidade de novas transferências de dados

Single Page Application (SPA)

Vantagens

- Experiência do usuário mais fluida e responsiva
- Menor tempo de resposta após o carregamento inicial
- Gerenciamento de estado mais complexo em grandes aplicações

Single Page Application (SPA)

Desvantagens

- Tempo de carregamento cresce com a complexidade da aplicação
- Dificulta *Search Engine Optimization* (SEO)

Single Page Application (SPA)

Quando usar?

- *Dashboards* e aplicações mais interativas
- Evitar utilizar para aplicações que dependem de SEO, ou seja que possuem conteúdo intensivo

Server-Side Rendering (SSR)

- Os clientes recebem uma página já renderizada no servidor
- A renderização ocorre no servidor antes de chegar ao navegador, após consultas a bancos de dados, APIs, etc
- Alguns elementos podem ser armazenados em *cache*

Server-Side Rendering (SSR)

Vantagens

- As alterações de conteúdo são exibidas instantaneamente, diferentemente dos SSGs, em que as equipes precisam reconstruir o site para visualizar as mudanças no conteúdo.
- O SSR permite que as equipes criem experiências de conteúdo dinâmicas e personalizadas sem soluções alternativas trabalhosas.
- Facilita a classificação por motores de busca (SEO)

Server-Side Rendering (SSR)

Desvantagens

- Normalmente, SSR exige mais chamadas de API para o servidor
- Por padrão, SSR costuma ser mais lento do que os SPAs e SSGs

Server-Side Rendering (SSR)

Quando aplicar?

> Ideais para experiências personalizadas em que as alterações dos dados podem ser vistas em tempo real. Aplicações SSR são opções excelentes para conteúdo sensível ao tempo e aplicações baseadas em interação intensiva.

Static Site Generator (SSG)

- Geram conteúdo no momento da criação de novas páginas ou quando alterações são feitas no conteúdo
- Não há necessidade de carregar páginas com base em solicitações do usuário

Static Site Generator (SSG)

Vantagens

- Criação do conteúdo desacoplada da arquitetura
- Carregamento rápido
- Melhor para otimização de busca (SEO)

Static Site Generator (SSG)

Desvantagens

- Dificuldade para aplicar conteúdos dinâmicos
- Mudança de conteúdo requer recriar o site (*rebuild*)

Static Site Generator (SSG)

Quando usar?

- Sites estáticos
- CDN (*Content Delivery Network*)

Frameworks

React

Single Page Application

> React é usado para criar uma aplicação que roda inteiramente no navegador do cliente após o carregamento inicial.

React

Single Page Application

- Funcionamento: O servidor envia um arquivo HTML praticamente vazio com uma div root. Todo o conteúdo da interface é gerado e gerenciado no lado do cliente (client-side) usando JS
- Ferramentas comuns: As ferramentas recomendadas são o Vite (que substituiu o antigo Create React App) ou o modo "SPA" de frameworks mais completos, como o React Router (antigo Remix)
- Quando usar: Ideal para aplicações internas, dashboards administrativos ou qualquer projeto onde o SEO (otimização para mecanismos de busca) não é um fator crítico e a experiência rica e interativa no cliente é prioridade .

React

Server-Side Rendering

> Nesta arquitetura, o React é executado no servidor para gerar o HTML completo da página, que é então enviado ao cliente.

React

Server-Side Rendering

- Funcionamento: O servidor (geralmente Node.js) usa funções como `renderToString` para transformar os componentes React em HTML
- Cada HTML é renderizado imediatamente no navegador, proporcionando uma sensação de carregamento mais rápido. Em seguida, o código JS do React é carregado e assume o controle no processo chamado de hidratação (`hydration`), tornando a página interativa

React

Server-Side Rendering

- Ferramentas comuns: A implementação manual de SSR com React e Node.js é possível, mas complexa. Por isso, a própria documentação do React recomenda o uso de frameworks que já incorporam essa funcionalidade, como o Next.js (na sua arquitetura Pages Router) ou o React Router (com SSR ativado)
- Quando usar: Essencial para sites de conteúdo, blogs, e-commerces e qualquer aplicação que dependa de um bom SEO e de um carregamento rápido da primeira página para uma melhor experiência do usuário

Abordagem Híbrida

Frameworks Modernos

> *Frameworks* como Next.js e React Router (antigo Remix) são frequentemente chamados de "híbridos" porque combinam o melhor dos dois mundos

Abordagem Híbrida

Frameworks Modernos

- A primeira visita a uma página pode ser feita com SSR, servindo HTML completo para carregamento rápido e SEO
- A navegação entre páginas subsequente acontece no cliente, como em uma SPA, sem recarregar a página inteira, o que torna a navegação muito fluida

Referências

- 5 Design Patterns on the Frontend
- Guide to modern frontend architecture patterns
- About React and Next.js

José Roberto Bezerra

jbroberto@ifce.edu.br

jbroberto76

pw1-repo

Powered by Slidev

Cover image by haikei